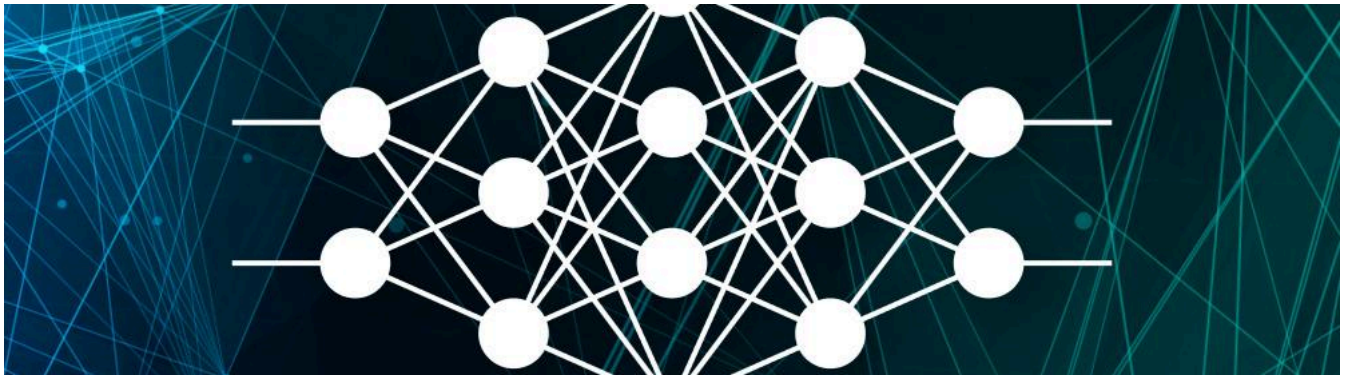




Deep Learning – Neural Network Fundamentals

1. What is a Neural Network?

A Neural Network is a **function approximator** that learns patterns from data by adjusting weights through training.

It mimics how the brain processes signals but works using **data and math, not biology**.

Interview-ready line:

“A neural network learns a mapping from inputs to outputs by optimizing weights to minimize loss.”

2. Core Components

Component	Meaning
Neuron	Small processing unit that computes a value
Weight	Importance assigned to each input

Bias	Extra adjustable value added for flexibility
Layer	Group of neurons working together
Loss	How wrong the model is
Optimizer	How the model updates weights

3. Architecture Flow (Forward Pass)

- Input → Hidden Layers → Output → Prediction

Step-wise explanation:

1. Input features are passed to neurons
2. Each neuron multiplies inputs by weights
3. Adds bias
4. Passes result to an activation function
5. Sends output to next layer
6. Final layer produces prediction

Interview-ready phrasing:

“In forward pass, data flows left to right, transforming into a prediction through weighted computation and non-linear activation.”

4. Activation Functions (Must Know for Interviews)

♦ Sigmoid

- Output range: 0 to 1
- Used when probability is needed

Interview question:

Why not use Sigmoid in deep networks?

Answer: Because it causes **vanishing gradients**.

♦ Tanh

- Output range: -1 to 1
- Zero-centered, stronger than sigmoid

Interview question:

Tanh vs Sigmoid?

Answer: Tanh is zero-centered and usually better for hidden layers.

♦ ReLU

- Most used activation today
- Keeps positive values, removes negatives
- Helps gradients flow better

Interview question:

Why ReLU is popular?

Answer: It's fast and reduces vanishing gradient issues.

♦ Softmax

- Used for **multiclass classification**
- Converts outputs into class probabilities that sum to 1

Interview-ready line:

“Softmax turns raw scores into normalized probabilities for multiple classes.”

5. Loss Functions (Must Know)

♦ **MSE (Mean Squared Error)**

- Used for regression
- Penalizes large errors more

Interview question:

Why MSE squares the error?

Answer: To penalize big mistakes more strongly.

♦ **Cross-Entropy Loss**

- Used for classification
- Measures difference between predicted probabilities and true labels

Interview-ready line:

“Cross-entropy measures how well predicted probabilities match actual labels.”

6. Optimizers (Intuition Only – No Need for Math)

Optim ize r

Why it exists

SGD	Basic optimizer, updates using gradient
Mome ntu m	Helps move faster in correct direction
RMSpr op	Stabilizes learning by controlling gradient scale
Adam	Combines Momentum + RMSprop, best default optimizer

Interview-ready line:

“Adam is the most used optimizer because it’s stable, fast, and adapts learning per parameter.”

7. Key Interview Concepts

Epoch vs Batch

- **Epoch:** One full pass over data
- **Batch:** Small subset passed at a time

Interview-ready explanation:

“We use batches to train faster and update weights multiple times per epoch.”

Why Neural Networks need activation?

Answer interview-ready:

“Without activation, all layers collapse into a linear function – activation adds non-linearity so the model can learn complex patterns.”

8. Advantages of Neural Networks

- Learn non-linear relationships
- Scale to large data
- Foundation for NLP, vision, and GenAI

9. Limitations

- Need more data than ML models
 - Harder to interpret
 - Can have vanishing/exploding gradients
-

10. Model Evolution Understanding (Interview Secret Weapon)

- ML models → Neural Networks → CNN/RNN/LSTM → Attention → Transformers → GenAI

Interview-ready insight:

“Neural networks are the base that enabled sequence models (LSTM/GRU), which later evolved into attention and transformers powering modern GenAI.”

11. Real-World Example

Imagine a company predicting loan approval probability:

- Inputs: income, age, credit score
 - Output layer uses **sigmoid**
 - Loss: **cross-entropy**
 - Optimizer: **Adam**
-



Final Interview-Ready Summary (Save This)

"A neural network consists of neurons that apply weighted computation + bias, pass results through activation, measure error using loss, and update weights using optimizers. It adds non-linearity through activation functions, and training stability through smart optimizers like Adam."

-



MODEL – Convolutional Neural Network (CNN)

(Interview-Perfect Notes | Vision + GenAI-Bridge)

1. What problem does CNN solve?

CNN is designed to:

- **Extract meaningful patterns from images**
- Handle **spatial relationships** (neighboring pixels matter)
- Learn features like **edges** → **textures** → **objects**

Industry examples:

- Face recognition
- Medical image analysis
- Object detection
- Image preprocessing in multimodal GenAI

Interview line:

“CNN automatically learns spatial image features using filters instead of manual feature engineering.”

2. Core intuition (no math, pure clarity)

“CNN scans an image using small learnable filters (kernels) to detect patterns. Early layers detect simple things (edges), later layers detect complex objects (faces, cars, text, etc.).”

3. CNN Architecture – step-by-step data flow

Input Image

- Convolution Layer (filters extract features)
- Activation (ReLU adds non-linearity)
- Pooling (reduces size, keeps important info)
- Repeat (deeper layers learn higher features)
- Fully Connected Layer (final reasoning/classification)
- Output

4. Key concepts interviewers test

◆ Filters / Kernels

- Small matrices that **slide over the image**
- Each filter learns to detect one type of pattern
- **Output = feature map**

Interview Q:

What is a feature map?

Answer:

"A feature map is the output image after applying a filter, highlighting detected patterns."

◆ Stride

- How many pixels the filter moves at a time

Interview Q:

Effect of high stride?

Answer:

"Feature map becomes smaller, computation faster, but can miss details."

◆ Padding

- Adds border pixels to preserve size

Interview Q:

Why padding is used?

Answer:

"To prevent losing corner/border information."

♦ **Pooling (Max / Avg)**

- Downsamples feature maps
- Reduces computation
- Controls overfitting

Interview Q:

Max Pool vs Avg Pool?

Answer:

"Max pooling keeps strongest feature, Avg pooling keeps overall signal."

♦ **ReLU Activation**

- Removes negative values
- Helps gradient flow

Interview Q:

Why ReLU after convolution?

Answer:

"To introduce non-linearity and avoid vanishing gradients."

5. Advantages

- Learns features automatically

- Handles non-linear image patterns
 - Efficient for vision tasks
 - Works well when spatial locality matters
-

6. Limitations

- Needs large data
 - Computationally heavy (vs ML models)
 - Less interpretable than ML models
 - Not ideal for **long text sequences** (RNN/Transformers better)
-

7. Important hyperparameters

- Number of filters
 - Filter size (3×3, 5×5, 7×7...)
 - Stride
 - Padding type
 - Pooling size
-

8. When to use CNN

- Image data

- Spatial feature learning
- Multimodal GenAI pipelines
- OCR preprocessing tasks

9. When NOT to use CNN

- Pure tabular data
- Very long sequential text
- When low-latency is mandatory

10. Comparison with similar architectures

Model	Handles Spatial Data	Feature Learning
kNN	✗ No	✗ Manual
Decision Tree	✗ No	✗ Manual
CNN	✓ Yes	✓ Automatic

Interview one-liner:

“kNN/trees memorize or split, CNN **extracts spatial features.**”

11. How CNN filters affect images (Interview storytelling insight)

If asked:

“What did you observe when applying filters in CNN?”

Say:

“Early filters highlight **edges and simple textures**, deeper filters highlight **parts of objects**, final filters capture **full object shapes**. This helps model understand structure hierarchically.”

12. Real-world example you can say in interviews

Imagine building a medical diagnosis system:

1. Input = MRI scan
 2. Multiple filters detect edges, shapes, anomalies
 3. ReLU removes noise
 4. Pooling reduces size
 5. Final dense layer makes classification decision
-



Final 15-second answer to save

“CNN extracts spatial image features using sliding filters, applies ReLU for non-linearity, pooling for compression, and fully connected layers for final reasoning. It learns features hierarchically from edges to objects.”



MODEL 23 – Recurrent Neural Network (RNN)

1. What problem does RNN solve?

RNN is built for **sequential data**, where **order and past context matter**.

Industry examples:

- Sentence modeling
- Speech recognition
- Stock or sensor sequences (univariate/multivariate temporal)
- Language modeling before Transformers

Interview line:

“RNN predicts by remembering past information using a recurrent hidden state.”

2. Core intuition (simple & clear)

“Unlike CNN or tree models, RNN has a **memory-like hidden state** that carries information from previous steps to the next step.”

In simple words:

“It reads the sequence step by step and updates what it remembers each time.”

3. How RNN works (step-by-step data flow)

Input (sequence token/image/signal)

↓

Hidden State stores previous context

↓

Loop runs for each time step

↓

Output generated at each step or final step

More detailed steps:

1. Take first input
 2. Compute hidden state
 3. Pass next input + previous hidden state together
 4. Update hidden state again
 5. Repeat until sequence ends
 6. Produce output
-

4. Key concepts interviewers test

◆ Hidden State

- Carries memory forward
- Updated at every step

Interview Q:

What does RNN remember?

Answer:

"It remembers a compressed representation of past inputs in the hidden state."

◆ Recurrent Connection

- Same weights used at every time step

Interview Q:

Why same weights?

Answer:

“So the network learns a consistent pattern over time and reduces parameters.”

♦ **Vanishing Gradient Problem (Very important!)**

Interview Q:

What is vanishing gradient in RNN?

Answer:

“When sequence is long, gradient becomes too small, so the model forgets early context – this led to LSTM/GRU and later Transformers.”

5. Advantages

- Handles order-dependent data
 - Shares weights → fewer parameters
 - Can process variable-length sequences
 - Good baseline for temporal patterns
-

6. Limitations

- Cannot remember long context well (vanishing gradients)
- Sequential → slow training (cannot fully parallelize)
- Less accurate than Transformers for language
- Hidden state is limited compared to attention

Interview one-liner:

“RNN is good for sequences, but not great for **long-term dependencies** – that’s why LSTM/GRU and then Transformers replaced it.”

7. Important hyperparameters

- Hidden state size
 - Sequence length used for training
 - Learning rate & weight initialization
-

8. When to use RNN

- Short-to-medium length sequences
 - Real-time streaming signals
 - Quick baselines for sequential trends
-

9. When NOT to use RNN

- Long documents or long sentences
 - When training speed matters (sequential bottleneck)
 - When relationships are global, not local
 - When LSTM/GRU/Transformers available (better options)
-

10. Comparison with related models

Model	Long Context Memory	Parallel Training	Main Strength
CNN	✗ No	✓ Yes	Spatial patterns
RNN	⚠ Medium	✗ No	Short sequences
LSTM/GRU	✓ Better	✗ Still sequential	Long dependency
Transformers	★ Best	★ Best	Global context via attention

11. Real-world example

Predicting next word in a sentence:

- RNN reads: "I am learning AI"
 - Learns context in hidden state
 - Predicts next word: "fast"
 - Loss optimized, weights updated, state carried forward
-

12. Why RNN matters for GenAI interviews

Because it builds intuition for:

- **Sequence modeling**
- **Memory & state**
- **Why gates were needed**
- **Why attention replaced recurrence**

Interview power line:

“RNN introduced the idea of stateful sequence memory, but attention scaled it globally.”



Final 20-second answer to save

“RNN processes sequential data step-by-step, maintaining a hidden state that carries context forward. It uses the same weights across time steps, which reduces parameters but struggles with long-term dependencies due to vanishing gradients. This limitation led to gated RNNs (LSTM/GRU) and eventually Transformers.”



MODEL 24 – Long Short-Term Memory (LSTM)

(Interview-Perfect Notes | Sequence + Memory Core)

1. What problem does LSTM solve?

LSTM solves the **long-term dependency problem** in sequences – where normal RNN fails due to **vanishing gradients** (model forgets old context).

Industry examples:

- Chatbots (intent memory)
- Speech to text
- Text generation (before transformers)

- Time-series forecasting
- Sequence labeling

Interview line:

“LSTM remembers long context using gated memory instead of a simple hidden state.”

2. Core intuition (crystal clear)

“LSTM has a **cell state** – a highway that carries memory forward. Gates decide what to **keep, forget, or output.**”

Think of it like:

- 🗑️ Forget useless memory
 - 📝 Store useful memory
 - 📦 Share required memory
-

3. Architecture & Data Flow

Input + Previous Hidden State

↓

Forget Gate → removes irrelevant past info

↓

Input Gate → stores important new info

↓

Cell State → carries long-term memory forward

↓

Output Gate → decides final output info

↓

Hidden State updated → passed to next step

4. The 3 Gates (Interviewers test this heavily)

♦ Forget Gate

Decides what past information to discard.

Interview answer:

“It removes memory that doesn’t help future prediction.”

♦ Input Gate

Decides what new information to store.

Interview answer:

“It writes new useful memory into the cell state.”

♦ Output Gate

Decides what information to send as output.

Interview answer:

“It reads filtered memory from the cell state to produce the final result.”

5. Key internal components

Component	Meaning
Cell State	Long-term memory highway
Hidden State	Short-term working memory

Gates

Controllers that regulate
memory

6. Advantages

- Remembers long sequences
 - Reduces vanishing gradients
 - Good for sequence-based reasoning
 - More stable than RNN
-

7. Limitations

- Still sequential (not parallel like transformers)
- Slow training
- More parameters → needs more compute
- Struggles with **very long global context** (Transformers better)

Interview one-liner:

“LSTM is strong for long sequences, but Attention scales globally – that’s why Transformers replaced it for LLMs.”

8. When to use LSTM

- When sequence is long
- When temporal dependency matters

- When labels are available for sequences
 - Baselines before Transformer adoption
-

9. When NOT to use LSTM

- When data is not sequential
 - When global relationships matter (use Attention/Transformers)
 - When low latency or parallelism is required
-

10. LSTM vs RNN vs Transformer

Model	Long Context	Parallel	Memory
RNN	✗ No	✗ No	Weak
LSTM	✓ Yes	✗ No	Strong gated
Transformer	★ Very strong	★ Yes	Uses attention + huge context

11. Why LSTM matters for GenAI interviews (even if not used today)

Because it helps you explain:

- Sequence memory evolution
- Vanishing gradients
- Why gating was needed

- Why attention is superior
- Temporal reasoning

Interview power line:

“LSTM introduced gated memory, and Transformers generalized it with global attention.”

12. Real-world example to explain in interviews

For a chatbot remembering conversation context:

- Forget gate → remove irrelevant chat history
 - Input gate → store important user intent
 - Cell state → maintain memory across dialogue
 - Output gate → generate the response
-

Final 20-Second Interview-Ready Summary

“LSTM is a gated recurrent network that maintains a long-term cell state. Forget, input, and output gates control memory flow, making it effective for long sequences.

It reduces vanishing gradients, but remains sequential and computationally heavier than Transformers, which use global attention and parallelism.”



MODEL 25 – GRU (Gated Recurrent Unit)

Interview-Perfect Notes | LSTM's Efficient Cousin

1. What problem does GRU solve?

GRU solves **long-term dependency** in sequences (like LSTM) but with a **simpler and faster architecture** and **fewer parameters**.

Used in industry for:

- NLP baselines
- Speech sequences
- Time-series models
- When compute efficiency matters

Interview line:

"GRU retains long-term context like LSTM but with a lighter architecture for faster training."

2. Core intuition (very clear)

"GRU has **2 main gates** that decide what to remember and what to forget. It merges memory and hidden state into one streamlined flow."

You can imagine:

-  **Update gate** → What to keep from past + new
 -  **Reset gate** → What past info to ignore
-

3. How GRU works (step-by-step)

Input + Previous State

↓

Reset Gate filters irrelevant past info

↓

Update Gate decides how much past + new info to keep

↓

New State created (merged memory + hidden)

↓

Output generated

4. GRU Gates (Interviewers test this)

♦ Reset Gate

- Filters out **irrelevant past context**

Interview answer:

“It decides how much past information to forget before combining with new input.”

♦ Update Gate

- Controls **past vs new memory balance**

Interview answer:

“It decides how much new info to add and how much past context to retain.”

5. Advantages

- Fewer parameters than LSTM → faster training
 - Better for low compute environments
 - Reduces vanishing gradient problem
 - Performs very well for many NLP tasks
-

6. Limitations

- Still sequential → not parallel like Transformers
- No separate cell state → slightly less control than LSTM
- Not best for **very long global context** (Transformers win here)

Interview one-liner:

“GRU is efficient but lacks global attention – Transformers scale context better.”

7. When to use GRU

- Long sequences
 - When speed + efficiency is priority
 - Baseline before trying attention/transformers
-

8. When NOT to use GRU

- Non-sequential data

- When global relationships or large context windows needed
-

9. GRU vs LSTM (Most asked comparison)

Aspect	LSTM	GRU
Gates	3	2
Parameters	More	Less (faster)
Memory control	Stronger	Simpler but efficient
Parallelism	✗ No	✗ No
Very long context	✗ Struggles	✗ Struggles

10. Why GRU matters for AI Engineer interviews?

Because it shows you understand:

- Sequence evolution
 - Memory efficiency
 - Gated architectures
 - Trade-offs in production baselines
-

Final 20-Second Interview Summary

“GRU is a gated recurrent network with **reset and update gates** that filter and balance past and new context. It handles long-term dependencies more efficiently than RNN and trains faster than LSTM due to fewer parameters.

However, it remains sequential and lacks global attention, which Transformers use to scale context.”

Attention Mechanism (Interview-Perfect Notes | 2026 AI Engineer Path)

1. Why Attention was invented?

Because **RNN/LSTM/GRU process sequentially and compress all memory into one vector**, they struggle to:

- Remember very long context
- Capture global relationships
- Parallelize training

Attention solved this by allowing the model to:

“Look back at all inputs and decide which part is most relevant for the current prediction.”

Interview line:

“Attention lets the model dynamically focus on relevant context instead of compressing it into one hidden state.”

2. Core intuition (crystal clear)

“Not all words/pixels/tokens are equally important.
Attention calculates **importance scores** and focuses more on high-score parts.”

Think of it like:

You don’t memorize an entire book page equally – you look at the most relevant lines when answering a question.

3. Key building blocks (Interviewers test this)

- ◆ **Query (Q)**

“What am I looking for?”

- ◆ **Key (K)**

“What do I contain?”

- ◆ **Value (V)**

“What information do I actually give if selected?”

4. How Attention works (Step-by-Step flow)

Input tokens

↓

Convert into Q, K, V vectors

↓

Compute similarity between Q and all K → get scores

↓

Apply softmax to normalize scores (attention weights)

↓

Multiply weights with V → get final context-aware output

↓

Send output to next layer

Interview-ready phrasing:

“Attention computes relevance using Q-K similarity, normalizes using softmax, and aggregates V to produce context-aware representations.”

5. How relevance (similarity) is computed?

Most commonly using:

- **Dot product**
- **Scaled dot product** (in transformers)
- **Cosine similarity** (for retrieval systems, not model attention)

Interview note:

“Attention is a ranking problem inside the model – similarity identifies importance.”

6. Types of Attention (Must Know)

1 Soft Attention

- Assigns probability weights to all tokens
- Differentiable → used in training

2 Self-Attention

- Tokens attend to each other within the same input
- Used in Transformers

3 Cross-Attention

- Decoder attends to encoder outputs (used in encoder–decoder transformers)
-

7. Advantages

- Captures global relationships
 - Parallelizable (unlike RNN)
 - No fixed memory bottleneck
 - Scales to long context windows (critical for LLMs)
-

8. Limitations

- Expensive for very long sequences (quadratic cost)
- Needs optimization (that’s why Flash Attention, sparse attention, KV caching exist – you can mention in interviews)

Interview one-liner:

“Attention improves context but increases computation – we optimize it in LLMs using caching and efficient attention.”

9. Real-world use case connection

Concept	Where it is used
Self-Attention	GPT, BERT, LLMs
Attention weights	Highlighting important tokens
Q-K similarity	Finding relevance

V aggregation	Context-aware output
Softmax weighting	Normalization

Interview Q&A

Q: RNN/LSTM already have memory, why attention?

A: Because RNN has **compressed memory**, Attention has **selective global memory**.

Q: What happens without attention?

A: The model becomes **blind to long-range relevance** and collapses into a sequential memory bottleneck.

Q: Why softmax is used?

A: To convert raw importance scores into **normalized probabilities**.

Q: Does attention require stationary data or scaling?

A: No – it works on raw embeddings, but transformers scale it for stability.

Final 20-Second Summary

“Attention converts inputs into Query, Key, and Value vectors. It compares Q with all K using dot product to compute relevance scores, normalizes them using softmax into attention weights, and aggregates V using those weights to generate context-aware outputs. It solved RNN’s memory bottleneck and enabled Transformers to scale, parallelize, and build modern LLMs and AI agents.”



MODEL 26 – TRANSFORMERS

Interview-Perfect Notes | GenAI + AI Engineer Path (2026 Ready)

1. Why Transformers were invented?

Before Transformers:

- **RNN/LSTM/GRU were sequential**
- Had **memory bottleneck**
- Training was **slow, not parallel**
- Couldn't capture **global token relationships**

Transformers solved this using:

“Global Attention + Parallel Processing + Scalable Context”

Interview line:

“Transformers replaced RNNs because they capture global dependencies and train in parallel using attention instead of recurrence.”

2. High-level architecture intuition

Transformers are made of:

- **Encoder** → understands input deeply
- **Decoder** → generates output step-by-step (autoregressive)

But modern LLMs like GPT use:

“Decoder-only architecture” (no separate encoder)

3. How data flows (Step-by-Step, No Confusion)

Input Tokens

↓

Embedding Layer (convert tokens → vectors)

↓

Positional Encoding (add order information)

↓

Self-Attention Layer (tokens interact globally)

↓

Feed-Forward Network (learn complex patterns)

↓

Repeat for N layers (deep reasoning)

↓

Output Layer (prediction/generation)

4. Most important internal components

◆ Embeddings

- Convert words into vector representations
- Capture meaning

Interview line:

“Embeddings represent semantic meaning; attention operates on embeddings, not raw tokens.”

◆ Positional Encoding

- Adds sequence order (since Transformer has no recurrence)

Interview Q:

What if we remove positional encoding?

Answer:

“The model loses order awareness – ‘I love AI’ and ‘AI love I’ would look the same.”

◆ **Self-Attention (Core Block)**

- Every token looks at all tokens and decides importance

Interview line:

“Self-attention computes Q-K similarity to rank relevance and aggregates V to build context-aware representations.”

◆ **Feed-Forward Network (FFN)**

- Applies non-linear transformation to learn complex features

Interview line:

“FFN learns feature interactions beyond attention.”

5. Key advantages

- Captures **global context**
- Training is **parallelizable**
- No fixed-size memory bottleneck
- Scales well for **long context windows**
- Best for **NLP and multimodal GenAI**

6. Key limitations

- Attention cost is **quadratic** (expensive for very long sequences)
- Optimized using:
 - KV caching
 - Efficient/Flash Attention
 - Sparse attention (you can mention this)

Interview line:

“Attention is expensive, but inference is optimized using caching and efficient attention.”

7. Transformer variants you must know

1 Encoder-only → BERT

- Understands input text deeply
- Used for classification, similarity, retrieval

2 Decoder-only → GPT, LLaMA, Claude, etc.

- Generates output autoregressively
- Used for GenAI and reasoning agents

3 Encoder-Decoder → T5, BART

- Used in translation, summarization, text transformation

8. Evolution understanding (interview secret weapon)

- ML Models
- Neural Networks
 - RNN/LSTM/GRU
 - Attention
 - Transformers
 - LLMs (GPT/BERT)
 - Agentic AI (ReAct, Tool Use, RAG, Memory)
-

9. Most asked interview comparisons

Model	Context Capture	Parallel	Architecture
RNN/LSTM/GRU	✗ Compressed, limited	✗ No	Recurrent
Transformer	★ Global, scalable	★ Yes	Attention-based
GPT	★ Very long causal context	★ Yes	Decoder-only
BERT	★ Bidirectional context	★ Yes	Encoder-only

Interview line:

“GPT predicts next token (causal), BERT predicts masked tokens (bidirectional).”

10. Real-world connection

Transformers are used in:

- ChatGPT, Gemini, Claude, LLaMA

- Semantic search
 - RAG pipelines
 - Tool-enabled AI agents
 - Multimodal models (vision + text)
-

Final 20-Second Interview-Ready Summary

“Transformers convert tokens into embeddings, inject order using positional encoding, compute global relevance using self-attention, refine reasoning using feed-forward layers, and generate predictions. They solved RNN’s sequential memory bottleneck and became the foundation for LLMs and agentic AI systems.”

GPT vs BERT – Deep Interview Master Notes

GPT (Generative Pretrained Transformer)

1. Model Type

- **Architecture:** *Decoder-only Transformer*
- **Nature:** *Autoregressive*
- **Goal:** *Next token prediction*

Interview line:

“GPT generates text by predicting one token at a time, based on past context.”

2. How GPT works (intuition first)

Input Prompt

↓

Tokenize

↓

Convert to embeddings

↓

Apply causal self-attention (looks only at past tokens)

↓

Feed-forward layers refine reasoning

↓

Predict next token

↓

Repeat until response ends

3. Key Strengths

- Text generation
- Reasoning tasks
- Conversational AI
- Tool-enabled agents (ReAct, planning, etc.)

- Industry default for GenAI products
-

4. Limitations

- Cannot see future tokens (causal attention)
 - Can hallucinate if knowledge is missing
 - Expensive without caching → solved using KV cache
-

5. Where GPT is used

- Chatbots, AI assistants
 - Content creation
 - Agentic AI reasoning loops
 - RAG + Tool calling
 - Code generation
 - Synthetic data generation
-

6. Common Interview Questions

Q: Why GPT uses decoder-only?

A: Because generation needs **causal flow (left-to-right)**, no future leakage.

Q: How GPT handles long context during inference?

A: Using **KV cache** to avoid recomputing attention for past tokens.

BERT (Bidirectional Encoder Representations from Transformers)

1. Model Type

- **Architecture:** *Encoder-only Transformer*
- **Nature:** *Bidirectional (sees full sentence at once)*
- **Goal:** *Masked token prediction*

Interview line:

“BERT understands text using left + right context simultaneously.”

2. How BERT works (intuition first)

Input Sentence

↓

Tokenize

↓

Random tokens masked (hidden)

↓

Convert to embeddings

↓

Self-attention interacts globally (bidirectional)

↓

Feed-forward layers refine understanding

↓

Predict masked tokens or sentence representation

3. Key Strengths

- Deep language understanding
 - Classification
 - Semantic similarity
 - Search and retrieval embeddings
 - Intent detection
 - Sentence representation for RAG pipelines
-

4. Limitations

- **Cannot generate text naturally**
 - Needs task-specific fine-tuning
 - Used for understanding, not creation
-

5. Where BERT is used

- Spam classification
 - Sentiment analysis
 - NER (Named Entity Recognition)
 - Question answering (extractive)
 - Embedding generation for search/RAG
-

6. Common Interview Questions

Q: Why BERT is bidirectional?

A: Because understanding needs **global context, not causal flow**.

Q: GPT vs BERT – main difference?

A:

- GPT → *Generates*
- BERT → *Understands*

“GPT is causal, BERT is masked and bidirectional.”

Head-to-Head Comparison (Interview Favorite)

Aspect	GPT	BERT
Architecture	Decoder-only	Encoder-only
Training Objective	Causal LM (next token)	Masked LM (fill blanks)
Context View	Past tokens only	Full sentence (L+R)
Main Use	Generation + reasoning	Understanding + embeddings
Text Output	Natural generation	 Not designed for generation
Cold Start in Recs	Can assist reasoning	Generates embeddings for similarity
Scalability	Needs caching for inference	Fast at inference
Industry Value	★ Most used for GenAI/Agents	★ Most used for Search/RAG

Interview-ready conclusion:

"In 2026 AI engineering, GPT drives reasoning & generation, BERT drives understanding & embedding space."

Interview Storytelling Tip

If interviewer asks:

"Transformers are many, why GPT & BERT specifically?"

Say:

"GPT shows **generation and reasoning** – needed for AI assistants and agents.
BERT shows **language understanding and embedding space** – needed for semantic search, intent detection, and RAG pipelines.
Together they cover 95% of real-world NLP requirements."

5 One-Liners to Save in Notes

1. "GPT is decoder-only and predicts next token to generate text."
2. "BERT is encoder-only and uses masked tokens to learn bidirectional understanding."
3. "GPT = generation + reasoning, BERT = understanding + embeddings."
4. "GPT uses causal attention, BERT uses masked bidirectional attention."
5. "For 2026 AI roles, GPT powers agents, BERT powers semantic retrieval."

Top 40 High-ROI NLP Interview Questions

Foundations

1. What is NLP and why is it important in AI systems?
2. Difference between NLU and NLG?
3. What is tokenization? Explain word vs subword tokenization.
4. What is stemming vs lemmatization? When would you choose each?
5. What is stop-word removal? When is it NOT useful?
6. Explain Bag-of-Words. Why is it limited?
7. Explain TF-IDF. When is it better than BoW?
8. What is the curse of dimensionality in NLP?
9. What are word embeddings? Why were they invented?
10. Cosine similarity vs dot product – when to use which?
11. What is the cold start problem in text recommenders?

RNN Era

12. Why were RNNs invented for NLP?
13. What is the vanishing gradient problem in RNNs?
14. LSTM vs GRU for NLP – tradeoffs?
15. Why RNN/LSTM/GRU struggle with long global context?

Attention & Transformers

16. What is attention in NLP?
17. Explain Query, Key, Value intuition.

18. What is self-attention?
19. Why do Transformers need positional encoding?
20. What makes Transformers faster than RNNs?
21. Encoder vs Decoder in Transformers?
22. Why GPT is decoder-only?
23. Why BERT is encoder-only and bidirectional?
24. Masked LM vs Causal LM – difference?
25. What is a context window? Why it matters?
26. How to handle long documents in Transformers?
27. What is KV caching and why is it used?

LLMs & Industry Stack

28. What is RAG (Retrieval Augmented Generation)? Why industry loves it?
29. How do you perform document chunking for RAG?
30. What is semantic search? How is it implemented?
31. What is NER (Named Entity Recognition)? Real world use?
32. How do LLMs hallucinate? How would you reduce it?
33. How would you evaluate an NLP model?
34. Precision vs recall in NLP – which matters for NER? Why?
35. What is log-loss / cross-entropy in text classification?
36. What is model drift in NLP? How to detect it?

- 37. How would you deploy an NLP model in production?
- 38. Explain tool-enabled LLM agents from NLP perspective.
- 39. What is intent classification? How does it work?
- 40. How do vector databases help NLP pipelines?
- 41. What is text generation latency bottleneck? How would you optimize?
- 42. What is leakage in NLP? How does ordered boosting in CatBoost avoid it? (bonus)

Practical & System Design

- 43. How filters/kernels detect features in text via CNN? When is it useful for NLP?
- 44. How would you build a spam classifier step by step?
- 45. If labels are limited, how would you use semi-supervised NLP?
- 46. How to handle imbalanced text classification?
- 47. What are explanation-first interview expectations in NLP?
- 48. How to create embeddings for custom domain data?
- 49. How to fine-tune BERT for classification?
- 50. How to fine-tune GPT/LLaMA for domain tasks?
- 51. How to prevent target leakage in NLP preprocessing?
- 52. Why transformer models outperform RNNs?
- 53. Would you choose SVM or BERT for text classification? Why?
- 54. How would you explain attention cost to business stakeholders?
- 55. How to debug poor model outputs in NLP?
- 56. How to version-control NLP models (MLOps intuition)?

